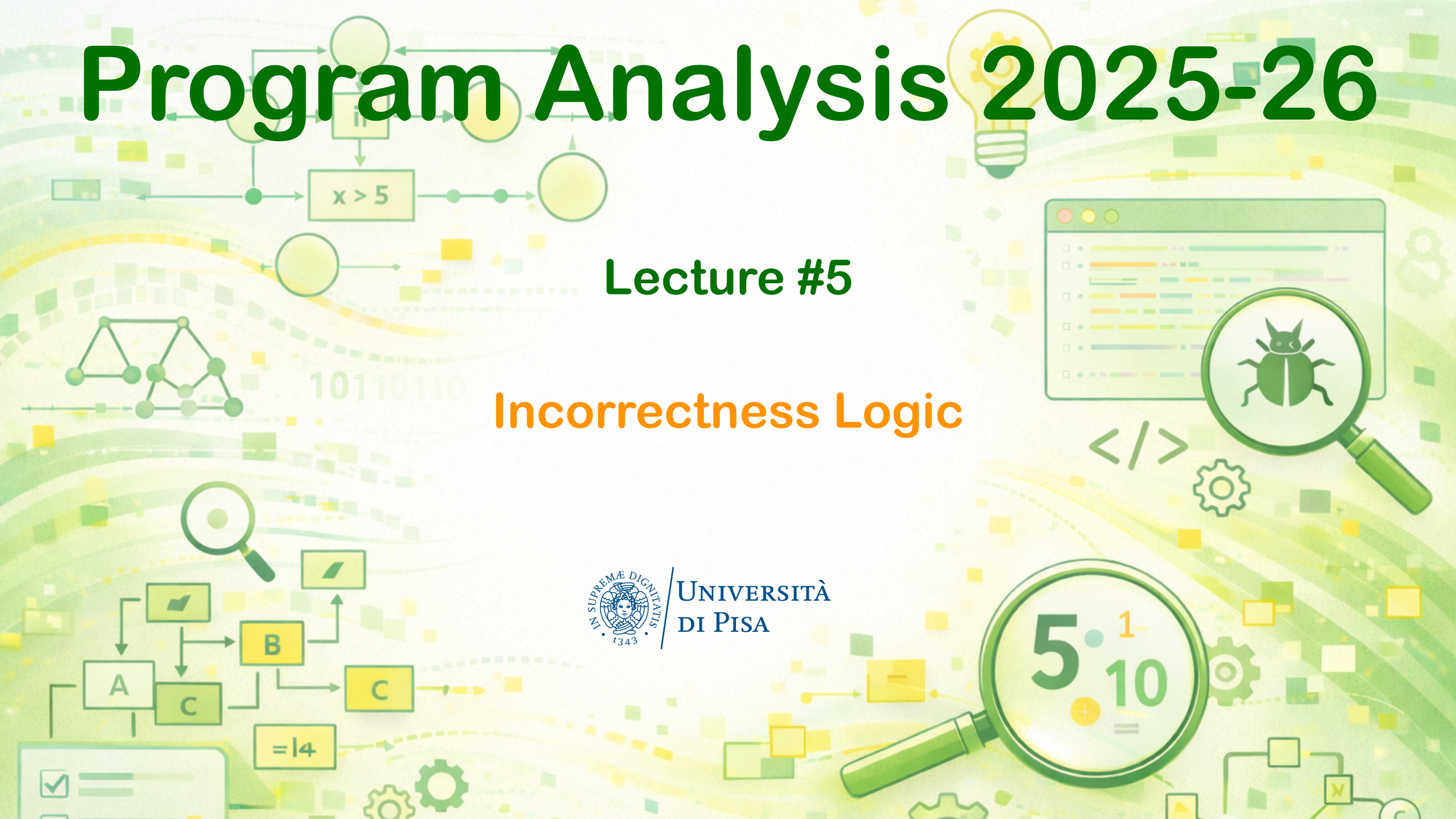
Program Analysis 2025-26

Lecture #5

Incorrectness Logic



Università di Pisa

Program incorrectness

POPL 2020

Incorrectness Logic

PETER W. O'HEARN, Facebook and University College London, UK

10

Program correctness and incorrectness are two sides of the same coin. As a programmer, even if you would like to have correctness, you might find yourself spending most of your time reasoning about incorrectness. This includes informal reasoning that people do while looking at or thinking about their code, as well as that supported by automated testing and static analysis tools. This paper describes a simple logic for program incorrectness which is, in a sense, the other side of the coin to Hoare's logic of correctness.

CCS Concepts: • **Theory of computation → Programming logic**.

Additional Key Words and Phrases: Proofs, Bugs, Static Analysis

ACM Reference Format:
Peter W. O'Hearn. 2020. Incorrectness Logic. *Proc. ACM Program. Lang.* 4, POPL, Article 10 (January 2020), 32 pages. https://doi.org/10.1145/3371078

1 INTRODUCTION

When reasoning informally about a program, people make abstract inferences about what might go wrong, as well as about what must go right. A programmer might ask "will the program crash if we give it a large string?", without saying *which* large string. In this paper we investigate the hypothesis that reasoning about the presence of bugs can be underpinned by sound techniques in a principled logical system, just as reasoning about correctness (absence of bugs) has been demonstrated to have sound logical principles in an extensive research literature. We also consider the relationship of the principles to automated reasoning tools for finding bugs in software.

We explore our hypothesis by defining incorrectness logic, a formalism that is similar to Hoare's logic of program correctness [Hoare 1969], except that it is oriented to proving incorrectness rather than correctness. Hoare's theory is based on specifications of the form

$$\{\textit{pre-condition}\}\textit{code}\{\textit{post-condition}\}$$

which say that the post-condition *over-approximates* (describes a superset of) the states reachable upon termination when the code is executed starting from states satisfying the pre-condition (the so-called strongest post). Conversely, we use a specification form

$$[\textit{presumption}]\textit{code}[\textit{result}]$$

which says that the post-assertion *result* be an under-approximation (subset) of the final states that can be reached starting from states satisfying the *presumption*.

The under-approximate triples were studied (with a different but equivalent definition) previously by de Vries and Koutavas [2011] in their reverse Hoare logic, which they used to specify randomized algorithms. Incorrectness logic adds post-assertions for errors as well as for normal termination, and these assertions describe erroneous states that can be reached by actual program executions. Dijkstra [1976] famously remarked that "testing can be quite effective for showing the presence of bugs, but is hopelessly inadequate for showing their absence," and he made this remark while arguing for the

Author's address: Peter W. O'Hearn, Facebook and University College London, UK.

2475-1421/2020/1-ART10
https://doi.org/10.1145/3371078

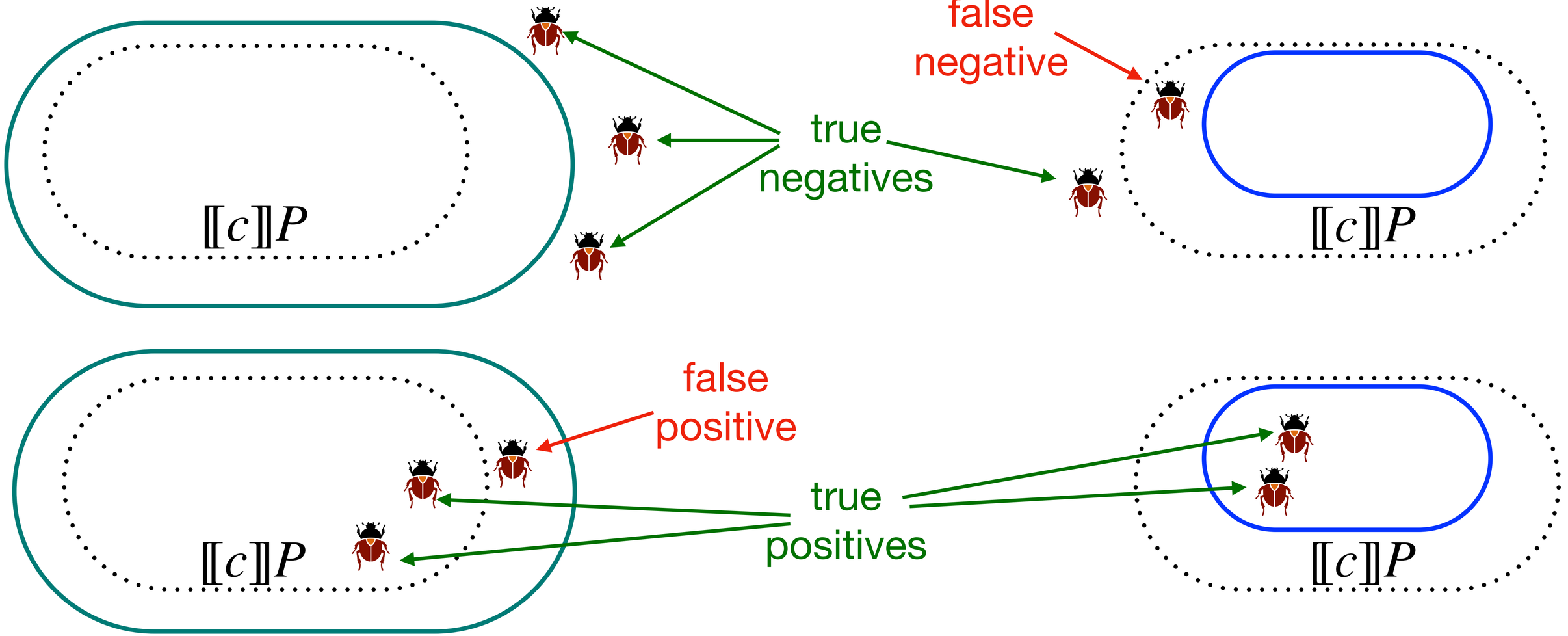
"Program correctness and incorrectness are two sides of the same coin"

Peter O'Hearn (2020)

Correctness vs incorrectness

Over-approximation:
good for proving correctness

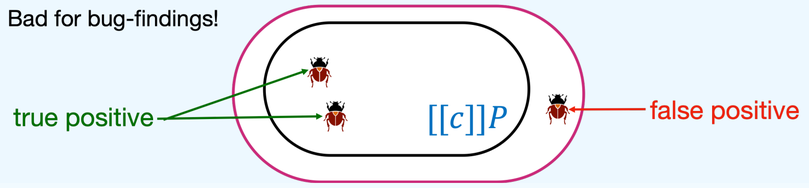
Under-approximation:
bad for proving correctness



bad for bug-finding

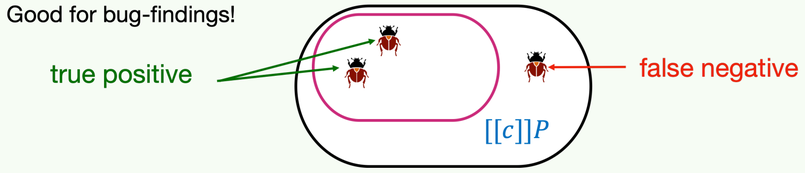
good for bug-finding

Hoare triples $\{P\}\ c\ \{Q\}$ iff $[\![c]\!]P \subseteq Q$



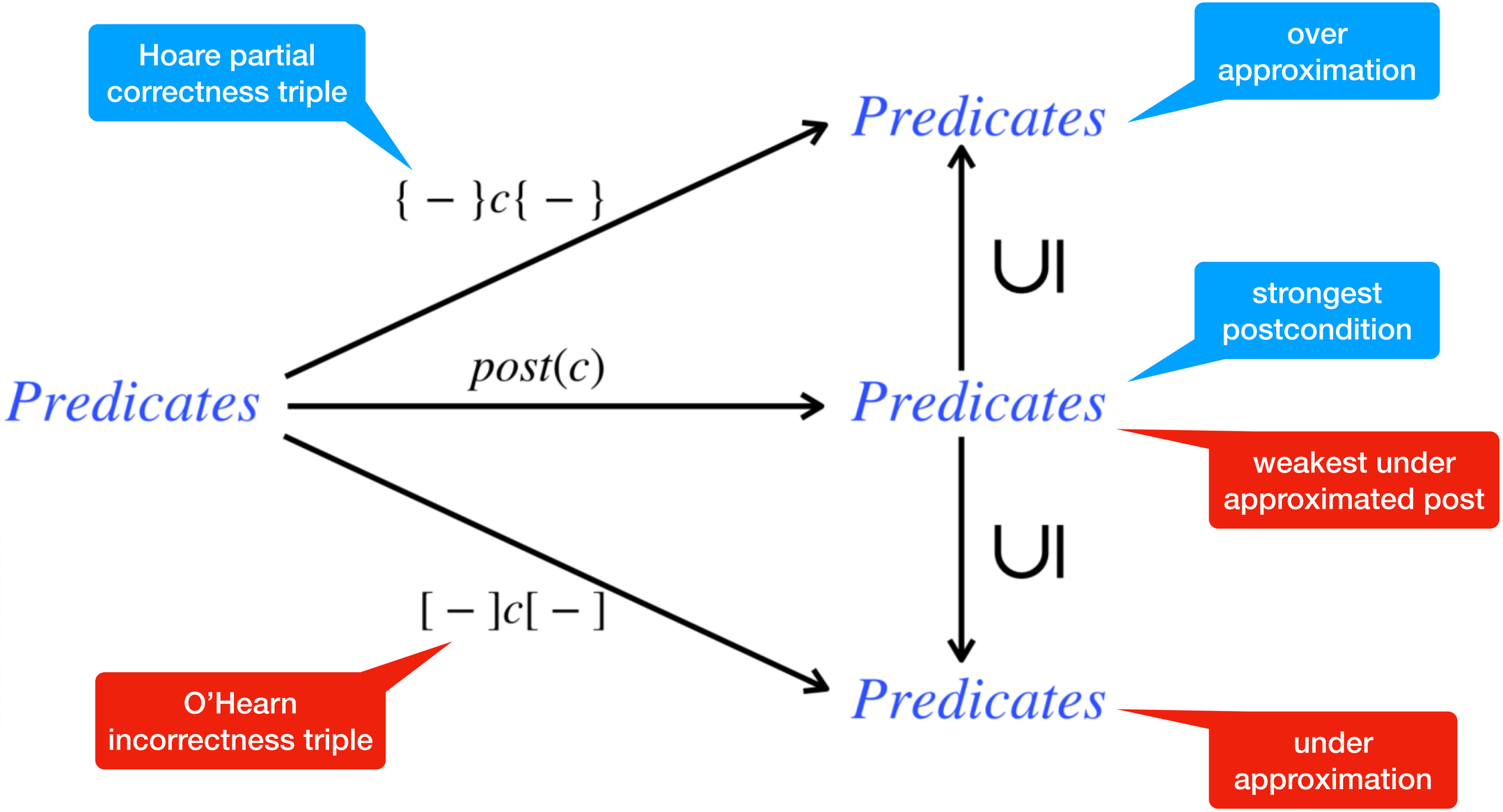
over approximation

IL triples $[P]\ c\ [Q]$ iff $[\![c]\!]P \supseteq Q$



under approximation

Picturing incorrectness



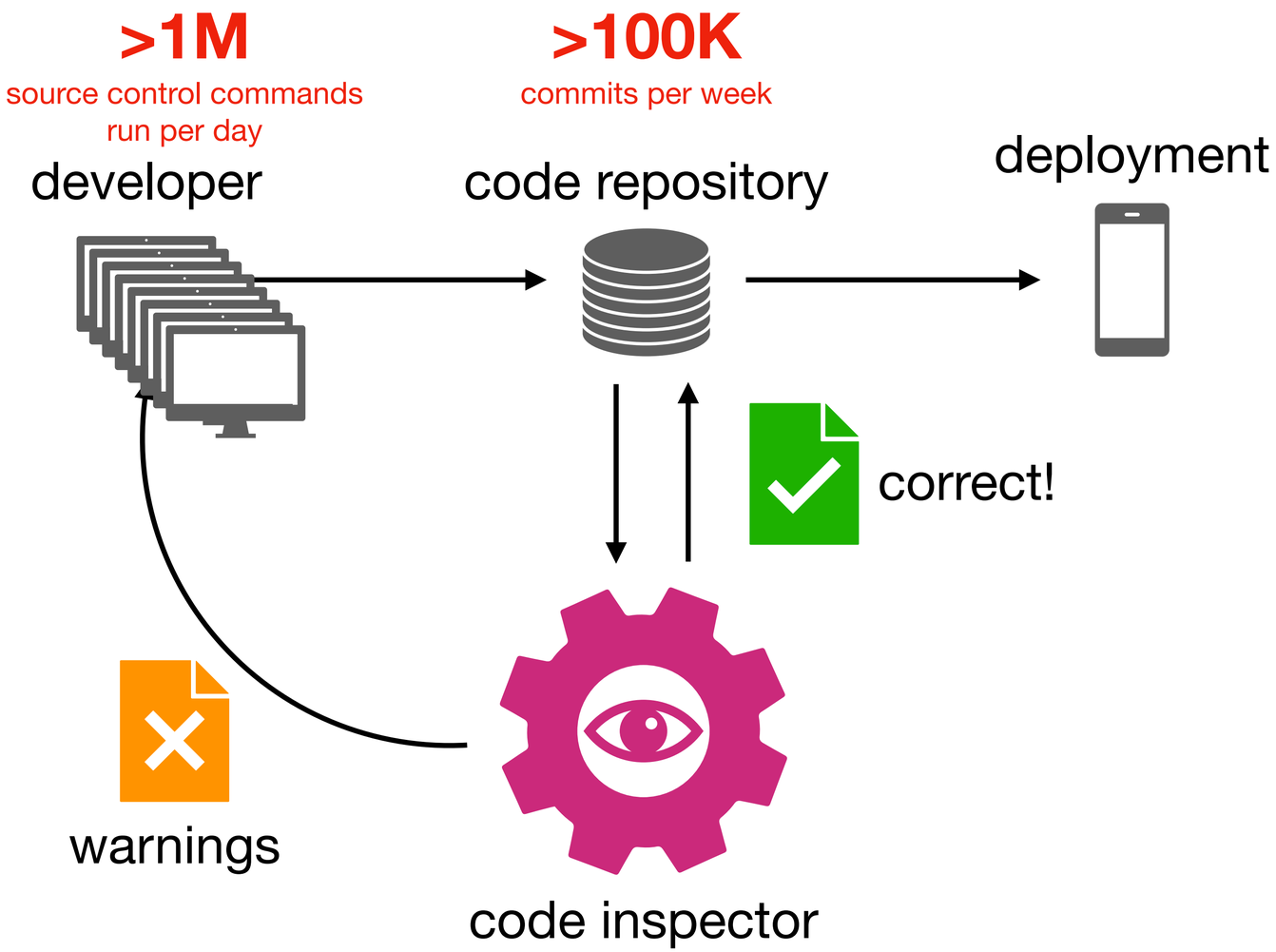
Recall

stronger $P \subseteq Q$ weaker

stronger $P \Rightarrow Q$ weaker

stronger $P \wedge Q \Rightarrow P \vee Q$ weaker

Correctness workflow, ideally



some scalability issues in a production environment:
analysis takes time (overnight?),
warnings are received late,
false positives mine credibility

Design principles

Low friction
do not rely on manual annotations

Act fast
able to report errors in less than 15’

Be compositional
whole program analysis is discouraged

Occam
do not use complex techniques (unless forced)

True positive theorem!
(under certain assumptions) the analyzer reports no false positives

“do not spam the developers!”

Incorrectness Logic (IL)

Hoare's triples

pre condition

$$\{P\}\ c\ \{Q\}$$

post condition

for any input matching the precondition
executing the command establishes the postcondition

$$[\![c]\!]P \subseteq Q$$

over approximation!

can include non reachable states

O'Hearn's triples

pre condition

$$[P]\ c\ [Q]$$

post condition

any output matching the postcondition
can be reached by executing the command
on some input matching the precondition

$$[\![c]\!]P \supseteq Q$$

under approximation!

includes just reachable states

As first order formulas

$\{P\}\ c\ \{Q\}$

$$[\![c]\!]P \subseteq Q \quad \equiv \quad \forall \sigma \in P\,.\,\forall \delta \in [\![c]\!]\sigma\,.\,\delta \in Q$$

Any reachable output satisfies the postcondition

$[P]\ c\ [Q]$

$$[\![c]\!]P \supseteq Q \quad \equiv \quad \forall \delta \in Q\,.\,\exists \sigma \in P\,.\,\delta \in [\![c]\!]\sigma$$

Any output in the postcondition is reachable

An example

$\{x < 0\}\ c\ \{x \geq 0\}$ if $x < 0$ initially, then $x \geq 0$ at the end

can c be $x := |x|$? can c diverge?

and $x := |x + 1|$? is $[x \mapsto 0]$ reachable?

and $x := 0$? can we produce all non-negative values?

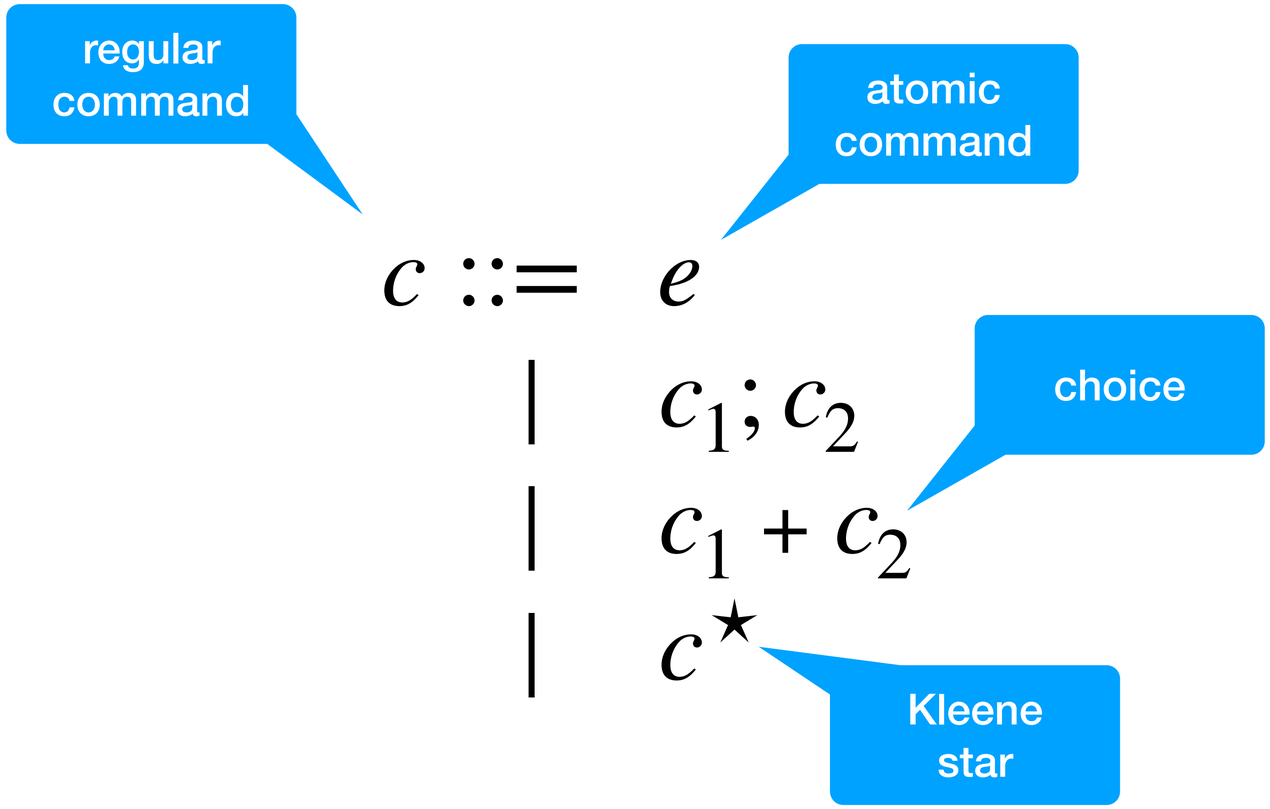
$[x < 0]\ c\ [x \geq 0]$ any $x \geq 0$ is reachable from some $x < 0$

can c be $x := |x|$? can c diverge?

and $x := |x + 1|$? is $[x \mapsto 0]$ reachable?

and $x := 0$? can we produce all non-negative values?

Regular commands



$$c ::= e \mid c_1; c_2 \mid c_1 + c_2 \mid c^\star$$

$$e ::= \text{skip} \mid x := a \mid b? \mid \text{error}() \mid x := \text{nondet}() \mid \ldots$$

$$\text{if } b \text{ then } c_1 \text{ else } c_2 \triangleq (b?; c_1) + (\neg b?; c_2)$$

$$\text{while } b \text{ do } c \triangleq (b?; c)^\star; \neg b?$$

Collecting / Relational semantics

$$[\![\texttt{error}()]\!]P \triangleq \varnothing \qquad [\![c]\!] : \wp(\Sigma) \to \wp(\Sigma)$$

$$[\![x := \texttt{nondet}()]\!]P \triangleq \{\ \sigma[v/x] \mid \sigma \in P, v \in \mathbb{Z}\ \}$$

$$[\![\texttt{error}()]\!] \triangleq \varnothing \qquad [\![c]\!] \subseteq \Sigma \times \Sigma$$

$$[\![x := \texttt{nondet}()]\!] \triangleq \{\ (\sigma, \sigma[v/x]) \mid \sigma \in \Sigma, v \in \mathbb{Z}\ \}$$

$$[\![\texttt{error}()]\!]^{\text{op}} \triangleq \varnothing$$

$$[\![x := \texttt{nondet}()]\!]^{\text{op}} \triangleq \{\ (\sigma[v/x], \sigma) \mid \sigma \in \Sigma, v \in \mathbb{Z}\ \}$$

Incorrectness logic rules

Floyd's axiom for assignment

[Floyd]
$$\frac{}{[P]\ x := a\ [\exists x' . P[x'/x] \land x = a[x'/x]]}$$

same as HL!

$$[y = 7]\ x := 42\ [\exists x' . y = 7 \land x = 42]$$

or more simply

$$[y = 7]\ x := 42\ [y = 7 \land x = 42]$$

Floyd's axiom for assignment

[Floyd] $$\frac{}{[P]\ x := a\ [\exists x' . P[x'/x] \land x = a[x'/x]]}$$

same as HL!

$$[y = 42]\ x := 42\ [x = y = 42]$$

would $[y = x]\ x := 42\ [x = y = 42]$ be valid as well?

Consequence rule

$$\frac{P' \Rightarrow P \quad [P']\ c\ [Q'] \quad Q \Rightarrow Q'}{[P]\ c\ [Q]}\ \text{[cons]}$$

shrink the post! (scalable bug detection)

$$[y = x]\ x := 42\ [x = 42] \Leftarrow [y = x = 42]$$

$$\frac{P \Rightarrow P' \quad \{P'\}\ c\ \{Q'\} \quad Q' \Rightarrow Q}{\{P\}\ c\ \{Q\}}\ \text{\{cons\}}$$

we can strengthen preconditions

we can weaken postconditions

Consequence rules explained

$\{P\}\ c\ \{Q\}$

can be weakened

UI

$[\![c]\!]P \subseteq Q$

UI

can be

strenghtened

$[P]\ c\ [Q]$

can be weakened

UI

$[\![c]\!]P \supseteq Q$

UI

can be

strenghtened

Some dualities

$$\text{[disj]}\ \frac{[P_1]\ c\ [Q_1] \quad [P_2]\ c\ [Q_2]}{[P_1 \vee P_2]\ c\ [Q_1 \vee Q_2]}$$

$\Rightarrow$ by rule [disj]

$\Leftarrow$ by rule [cons]

$$[P]\ c\ [Q_1] \ \wedge\ [P]\ c\ [Q_2] \quad \Leftrightarrow \quad [P]\ c\ [Q_1 \vee Q_2]$$

$$\{P\}\ c\ \{Q_1\} \wedge \{P\}\ c\ \{Q_2\} \quad \Leftrightarrow \quad \{P\}\ c\ \{Q_1 \wedge Q_2\}$$

$$\text{\{conj\}}\ \frac{\{P_1\}\ c\ \{Q_1\} \quad \{P_2\}\ c\ \{Q_2\}}{\{P_1 \wedge P_2\}\ c\ \{Q_1 \wedge Q_2\}}$$

$\Rightarrow$ by rule {conj}

$\Leftarrow$ by rule {cons}

Further dualities

dropping conjuncts by {cons} rule

$$\frac{\{P\}\ c\ \{Q \wedge R\}}{\{P\}\ c\ \{Q\}}$$

$$Q \wedge R \Rightarrow Q$$

dropping disjuncts by [cons] rule

$$\frac{[P]\ c\ [Q \vee R]}{[P]\ c\ [Q]}$$

$$Q \vee R \Leftarrow Q$$

Hoare's axiom for assignment?

$$\text{[Hoare]}\quad \frac{}{[Q[a/x]]\ x := a\ [Q]}$$

sound for HL
not for IL!

$$[y = 42]\ x := 42\ [y = x]$$

unsound!

The post-condition does **not** under-approximate the states reachable from the pre-condition

$\delta \triangleq [x \mapsto 3, y \mapsto 3] \vDash (x = y)$ is unreachable!

Other atomic commands

[skip] $\dfrac{}{[P]\ \text{skip}\ [P]}$

$\dfrac{}{[P]\ b?\ [P \wedge b]}$ [assume]

[error] $\dfrac{}{[P]\ \text{error}()\ [\text{false}]}$

$\dfrac{}{[P]\ x := \text{nondet}()\ [\exists x\,.\ P]}$ [nondet]

Sequence

$$\text{[seq]}\ \frac{[P]\ c_1\ [R] \qquad [R]\ c_2\ [Q]}{[P]\ c_1; c_2\ [Q]}$$

same as HL!

$$[x > 0]\ (y > x)?\ ;\ x := y\ [x = y > 1]$$

$$[y > x > 0]$$

Choice

$$\text{[choice]}\ \frac{[P]\ c_1\ [Q_1] \quad [P]\ c_2\ [Q_2]}{[P]\ c_1 + c_2\ [Q_1 \vee Q_2]}$$

$$[x > 0]\ \text{skip}\ +\ x := x + 1\ [x > 0]$$

$$[x > 0] \qquad [x > 1]$$

Choice + Consequence

$$\text{[choice]}\ \frac{[P]\ c_1\ [Q_1] \qquad [P]\ c_2\ [Q_2]}{[P]\ c_1 + c_2\ [Q_1 \vee Q_2]} \qquad \frac{(Q_1 \vee Q_2) \Leftarrow Q_2}{[P]\ c_1 + c_2\ [Q_2]}\ \text{[cons]}$$

$$[x > 0]\ \text{skip}\ +\ x := x + 1\ [x > 1]$$

$$[x > 0] \qquad\qquad [x > 1]$$

Choice: dropping disjuncts

[choiceL] $$\frac{[P]\ c_1\ [Q]}{[P]\ c_1 + c_2\ [Q]}$$

[choiceR] $$\frac{[P]\ c_2\ [Q]}{[P]\ c_1 + c_2\ [Q]}$$

sound under-approximation!
scalable bug detection

$$[y = x]\ (x := y - 1) + (x := y + 1)\ [y - x = 1]$$

$$[y = x]\ (x := y - 1) + (x := y + 1)\ [x - y = 1]$$

Example

$[y = 0]$ if $even(x)$ then $y := 42$ $[y = 42]$ 

is it a valid IL triple?

$$(y = 42) \triangleq \{\ [x \mapsto 0, y \mapsto 42],\ [x \mapsto 1, y \mapsto 42],\ [x \mapsto 2, y \mapsto 42],\ \ldots\ \}$$

All the states above satisfy the post $[y = 42]$ since it does not constrain x in any way.

In particular, the condition is satisfied by states like δ below, where x is odd

$$\delta \triangleq [x \mapsto 1, y \mapsto 42] \vDash (y = 42)$$

But δ is unreachable from any $\sigma \vDash (y = 0)$! Therefore the triple is not valid

Example

$[y = 0]$ if $even(x)$ then $y := 42$ $[y = 42 \wedge even(x)]$ 

is it a valid IL triple?

$(y = 42 \wedge even(x)) \triangleq \{\ [x \mapsto 0, y \mapsto 42],\ [x \mapsto 2, y \mapsto 42],\ \ldots\ \}$

IL vs HL

$[y = 0]$ if $even(x)$ then $y := 42$ $[y = 42 \wedge even(x)]$

Just seen

$\{y = 0\}$ if $even(x)$ then $y := 42$ $\{y = 42 \wedge even(x)\}$

For instance, $\sigma' = [x \mapsto 1, y \mapsto 0] \models (y = 0)$

$\{y = 0 \wedge even(x)\}$ if $even(x)$ then $y := 42$ $\{y = 42\}$

Note that also the following triple is valid, but we can drop the part $even(x)$:
$\{y = 0 \wedge even(x)\}$ `if` $even(x)$ `then y := 42` $\{y = 42 \wedge even(x)\}$

$[y = 0 \wedge even(x)]$ if $even(x)$ then $y := 42$ $[y = 42]$

$[y = 42]$ is too large and not sufficiently precise: we have forgotten that x is even. Also $\sigma' = [x \mapsto 1, y \mapsto 0] \models (y = 42)$

Derived rule for if

$$\text{if } b \text{ then } c_1 \text{ else } c_2 \triangleq (b?; c_1) + (\neg b?; c_2)$$

$$\text{[if]}\ \frac{[P \wedge b]\ c_1\ [Q_1] \qquad [P \wedge \neg b]\ c_2\ [Q_2]}{[P]\ \text{if } b \text{ then } c_1 \text{ else } c_2\ [Q_1 \vee Q_2]}$$

Just use [assume], [seq], [choice]

HL vs IL

For correctness reasoning

You **get to forget** information as you go along a path, but you **must remember** all the paths.

For incorrectness reasoning

You **must remember** information as you go along a path, but you **get to forget** some of the paths

Loop: bounded unrolling

$$\frac{}{[P]\ c^\star\ [P]}\ \text{[iter0]}$$

Any is P a valid under-approximation

$$\frac{[P]\ c^\star; c\ [Q]}{[P]\ c^\star\ [Q]}\ \text{[unroll]}$$

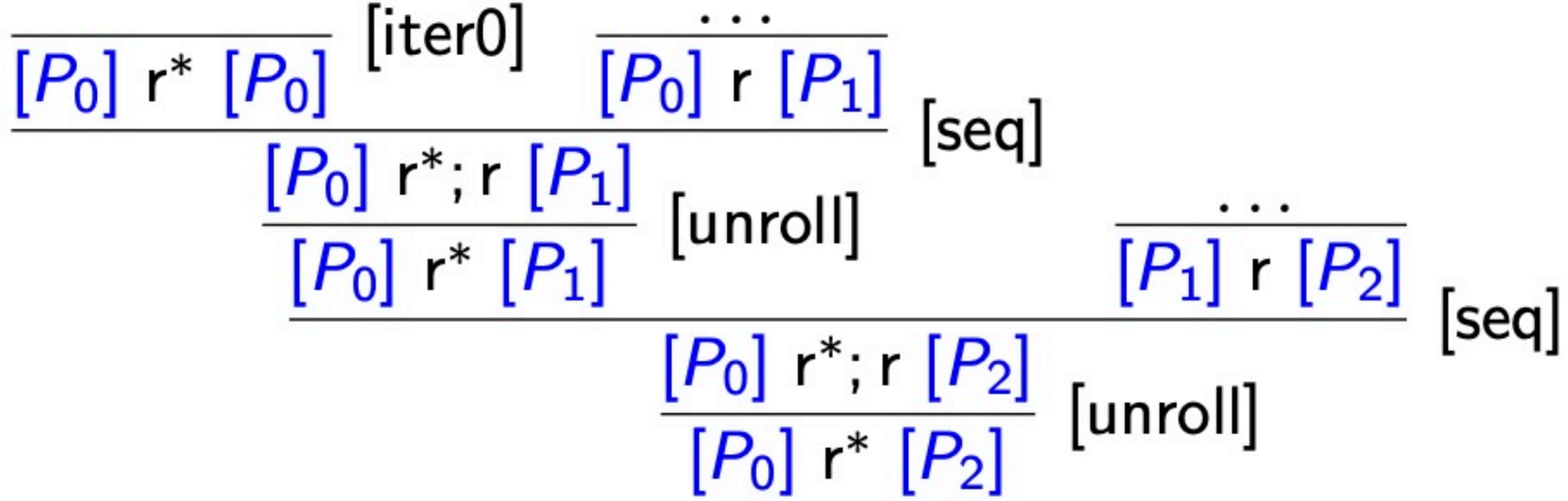
We can unroll to analyse any finite number of iterations

sound under-approximation!
scalable bug detection

$$[x = 0]\ (x := x + 1)^\star\ [x = 0]$$
$$[x = 0]\ (x := x + 1)^\star\ [x = 1] \qquad (x := x + 1)^\star ; x := x + 1$$
$$[x = 0]\ (x := x + 1)^\star\ [x = 2] \qquad (x := x + 1)^\star ; x := x + 1 ; x := x + 1$$

Loop: bounded unrolling (cont.d)

How do we unroll a finite number of times?



Key Idea: loop unrolling proves a loop triple by evaluating a finite number of iterations, without requiring invariants. It allows us to discover some post-conditions for a loop
Termination: reachability does not imply that a loop must terminate on all executions: only that enough paths terminate to cover all the states in the result assertion
Soundness: it is enough to show the result is reachable via some execution path, rather than reasoning about all possible loop behaviors

Backwards variant (weak)

$$\frac{\forall n \in \mathbb{N}.\ [P_n]\ c\ [P_{n+1}]}{[P_0]\ c^\star\ [P_k]}\ \text{[iter]}$$

[iter] has mostly a theoretical value.
In practice unrolling is most used

loop invariants are inherently over-approximations
sub-variants: to reason about loop under-approximation

$[x = 0]\ (x := x + 1)^\star\ [x = 2^{42}]$

$(x := x + 1)^\star\ ;\ \text{if}\ (x = 2^{42})\ \text{then error}()$

Backwards variant (weak)

$$\frac{\forall n \in \mathbb{N}.\ [P_n \equiv (x = n)]\ x := x + 1\ [P_{n+1} \equiv (x = n + 1)]}{[P_0 \equiv (x = 0)]\ (x := x + 1)^\star\ [P_{2^{42}} \equiv (x = 2^{42})]}\ \text{[iter]}$$

$$[x = 0]\ (x := x + 1)^\star\ [x = 2^{42}]\ \ /\!/\ P_n \triangleq (x = n)$$

Backwards variant

$$\frac{\forall n \in \mathbb{N}.\ [P_n]\ c\ [P_{n+1}]}{[P_0]\ c^\star\ [\exists k\,.\ P_k]}\ \text{[iter]}$$

$$\exists k\,.\ P_k \equiv \bigvee_{k=0}^{\infty} P_k$$

[iter] has mostly a theoretical value.
In practice unrolling is most used

$$[x = 1]\ (x := 2 \times x)^\star\ [\exists k\,.\ x = 2^k]$$

Backwards variant

$$\frac{\forall n \in \mathbb{N}.\ [P_n \equiv (x = 2^n)]\ x := 2 \times x\ [P_{n+1} \equiv (x = 2^{n+1})]}{[P_0 \equiv (x = 2^0)]\ (x := 2 \times x)^\star\ [\exists k.\ x = 2^k]}\ \text{[iter]}$$

$$[x = 1]\ (x := 2 \times x)^\star\ [\exists k.\ x = 2^k]\ \ // \ P_n \triangleq (x = 2^n)$$

Finite unrolling of while loops

$$\text{while } b \text{ do } c \triangleq (b?; c)^\star; \neg b?$$

$$\frac{\forall n \in \mathbb{N}.\ [P_n \wedge b]\ c\ [P_{n+1}]}{\forall n \in \mathbb{N}.\ [P_n]\ b?; c\ [P_{n+1}]}$$

$$\frac{\forall n \in \mathbb{N}.\ [P_n]\ b?; c\ [P_{n+1}]}{[P_0]\ (b?; c)^\star\ [\exists k\,.\ P_k]}$$

$$\frac{[P_0]\ (b?; c)^\star\ [\exists k\,.\ P_k]}{[P_0]\ \text{while } b \text{ do } c\ [\exists k\,.\ P_k \wedge \neg b]}$$

$$\frac{}{[P_0]\ \text{while } b \text{ do } c\ [P_0 \wedge \neg b]} \qquad \frac{[P_0 \wedge b]\ c\ [P_1]}{[P_0]\ \text{while } b \text{ do } c\ [(P_0 \vee P_1) \wedge \neg b]}$$

Principle of agreement

Theorem.

If $[P']\ c\ [Q']$

$\wedge\ \ P' \Rightarrow P$

$\wedge\ \{P\}\ c\ \{Q\}$

then $Q' \Rightarrow Q$

Proof.

Q'

$\subseteq [\![c]\!]P'$ // by IL

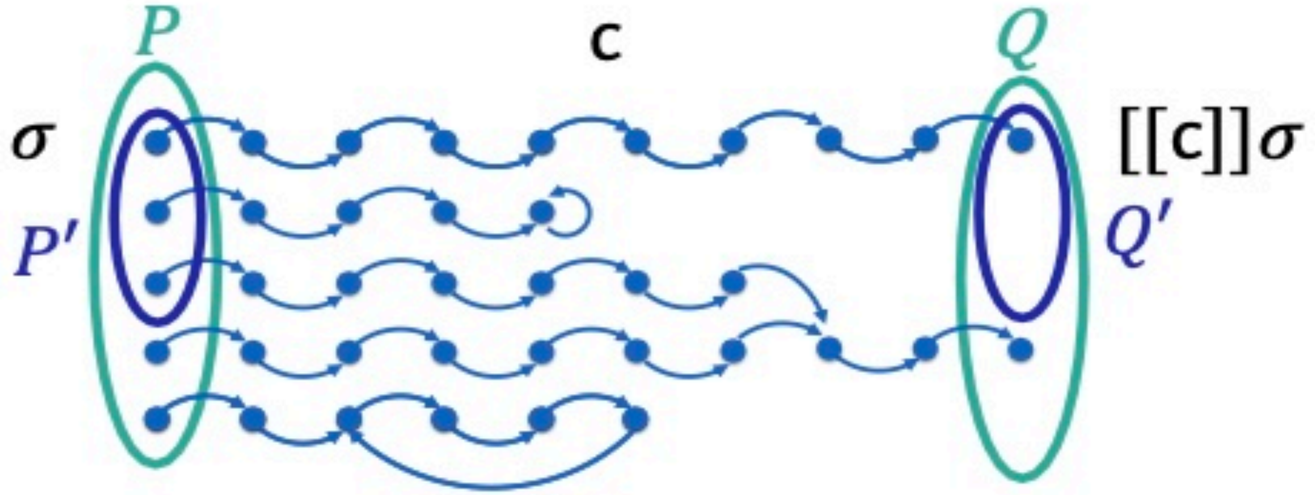
$\subseteq [\![c]\!]P$ // $P' \Rightarrow P$

$\subseteq Q$ // by HL

Pictorially

Principle of agreement

$$[P'] \; c \; [Q'] \wedge P' \subseteq P \wedge \{P\} \; c \; \{Q\} \Rightarrow Q' \subseteq Q$$



- $\{P\} \; c \; \{Q\}$ iff $[[c]]P \subseteq Q$
- $[P'] \; c \; [Q']$ iff $[[c]]P' \supseteq Q'$

Principle of denial

Theorem.

If $[P'] \; c \; [Q']$

$\wedge \quad P' \Rightarrow P$

$\wedge \; \{P\} \; c \; \{Q\}$

then $Q' \Rightarrow Q$

Cor. (by contraposition)

If $[P'] \; c \; [Q']$

$\wedge \quad P' \Rightarrow P$

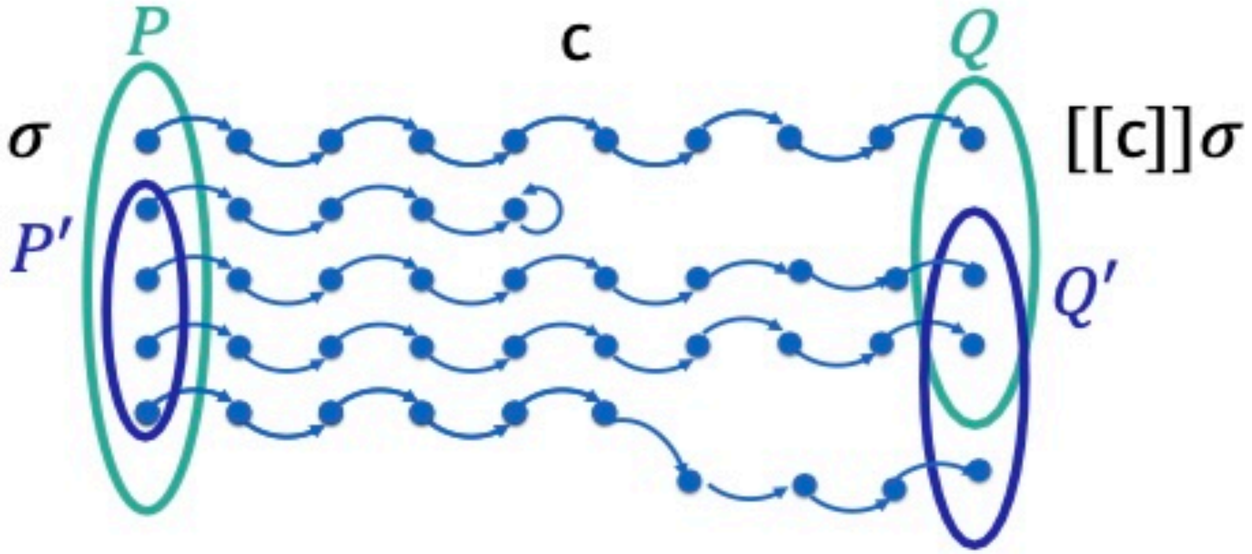
$\wedge \neg (Q' \Rightarrow Q)$

then $\neg (\{P\} \; c \; \{Q\})$

Pictorially

Principle of denial

$$[P'] \; c \; [Q'] \wedge P' \subseteq P \wedge \neg (Q' \subseteq Q) \Rightarrow \neg \{P\} \; c \; \{Q\}$$



- $\{P\} \; c \; \{Q\}$ iff $[[c]]P \subseteq Q$
- $[P'] \; c \; [Q']$ iff $[[c]]P' \supseteq Q'$

Examples

$[\text{true}]$
if $x \geq 0$ then
  $[x \geq 0]$
  skip
  $[x \geq 0]$
else
  $[x < 0]$
  $x := -x$
  $[\exists x' .\ x' < 0 \wedge x = -x'] \equiv [x > 0]$
$[(x \geq 0) \vee (x > 0)] \equiv [x \geq 0]$

$$\dfrac{\dfrac{}{[\text{true} \wedge x \geq 0]\ \texttt{skip}\ \ [x \geq 0]}\ [\text{skip}] \quad \dfrac{}{[\text{true} \wedge x < 0]\ \texttt{x:= -x}\ \ [x > 0]}\ [\text{assign}]}{\dfrac{[\text{true}]\ \texttt{if (x} \geq \texttt{0) then skip else x:= -x}\ [x \geq 0 \vee x > 0]}{[\text{true}]\ \texttt{if (x} \geq \texttt{0) then skip else x:= -x}\ [x \geq 0]}\ [\text{cons}]}\ [\text{if}]$$

Examples

$[z = 11]$
if $even(x)$ then
  $[z = 11 \wedge even(x)]$
  if $odd(y)$ then
    $[z = 11 \wedge even(x) \wedge odd(y)]$
    $z := 42$
    $[z = 42 \wedge even(x) \wedge odd(y)]$
  $[z = 42 \wedge even(x) \wedge odd(y)]$

The post-assertion corresponds to only one path through the program:
we can focus on it and ignore the paths corresponding to the else branches

Exercise

$[true]$

$n := \text{nondet}();$

$x := 0;$

while $(n > 0)$ do

$x := x + n;$

$n := \text{nondet}();$

$[?]$

Complete the triple and prove it